

WEEK11-SKILL

Task1:

To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components

```
UW PICO 5.09 File: integrated_shell.c Modified
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#define MAX_INPUT 200
#define MAX_ARGS 20

/* Parse the user's command */
int parse_command(char *input, char *args[])
{
    int count = 0;
    char *token = strtok(input, " ");
    while (token != NULL && count < MAX_ARGS - 1)
    {
        args[count] = token;
        count++;
        token = strtok(NULL, " ");
    }
    args[count] = NULL;
    return count;
}

/* Check whether the command is valid */
int validate_command(char *command)
{
    if (command == NULL)
    {
        return 0;
    }
    if (strlen(command) == 0)
    {
        return 0;
    }
    return 1;
}

/* Execute the command */
int execute_command(char *args[])
{
    pid_t pid = fork();
    if (pid == -1)
    {
        perror("fork");
        return -1;
    }
    if (pid == 0)
    {
        execvp(args[0], args);
    }
}
```

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell

```
UW PICO 5.09 File: integrated_shell.c Modified
int validate_command(char *command)
{
    if (command == NULL)
    {
        return 0;
    }

    if (strlen(command) == 0)
    {
        return 0;
    }

    return 1;
}

/* Execute the command */
int execute_command(char *args[])
{
    pid_t pid = fork();

    if (pid == -1)
    {
        perror("fork");
        return -1;
    }

    if (pid == 0)
    {
        execvp(args[0], args);

        /*
         * Reached only if execvp fails.
         */
        perror("exec");
        exit(127);
    }

    int status;

    if (waitpid(pid, &status, 0) == -1)
    {
        perror("waitpid");
        return -1;
    }

    if (WIFEXITED(status))
    {
        return WEXITSTATUS(status);
    }

    if (WIFSIGNALED(status))
    {
        printf("Process terminated by signal %d\n",
            WTERMSIG(status));
        return -1;
    }
}

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell
```

```
UW PICO 5.09 File: integrated_shell.c Modified

/*
 * Reached only if execvp fails.
 */
perror("exec");
exit(127);
}

int status;

if (waitpid(pid, &status, 0) == -1)
{
    perror("waitpid");
    return -1;
}

if (WIFEXITED(status))
{
    return WEXITSTATUS(status);
}

if (WIFSIGNALED(status))
{
    printf("Process terminated by signal %d\n",
        WTERMSIG(status));
    return -1;
}

return 0;
}

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("Integrated Shell\n");
    printf("Type 'exit' to quit.\n\n");

    while (1)
    {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
        {
            printf("Shell exiting...\n");
            break;
        }
    }
}

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell
```

```
UW PIC0 5.09 File: integrated_shell.c Modified
}
int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("Integrated Shell\n");
    printf("Type 'exit' to quit.\n\n");

    while (1)
    {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
        {
            printf("Shell exiting...\n");
            break;
        }

        if (strlen(input) == 0)
        {
            continue;
        }

        int argc = parse_command(input, args);

        if (argc == 0)
        {
            printf("Error: empty command\n");
            continue;
        }

        if (!validate_command(args[0]))
        {
            printf("Error: invalid command\n");
            continue;
        }

        int result = execute_command(args);

        if (result == 127)
        {
            printf("Command could not be executed.\n");
        }
    }

    return 0;
}

⌘ Get Help      ⌘ WriteOut    ⌘ Read File    ⌘ Prev Pg      ⌘ Cut Text     ⌘ Cur Pos
⌘ Exit          ⌘ Justify     ⌘ Where is    ⌘ Next Pg      ⌘ UnCut Text   ⌘ To Spell
```

```
Last login: Sun Sep 27 11:00:59 on ttys000
(vishalkarnati@VISHALS-MacBook-Air ~ % cd ~/Desktop/os
vishalkarnati@VISHALS-MacBook-Air os % mkdir week1skill
vishalkarnati@VISHALS-MacBook-Air os % cd week1skill
vishalkarnati@VISHALS-MacBook-Air week1skill % nano integrated_shell.c
vishalkarnati@VISHALS-MacBook-Air week1skill % gcc integrated_shell.c -o integrated_shell
vishalkarnati@VISHALS-MacBook-Air week1skill % ./integrated_shell
Integrated Shell
Type 'exit' to quit.

myshell> ls
integrated_shell      integrated_shell.c
myshell> wd
exec: No such file or directory
Command could not be executed.
myshell> pwd
/Users/vishalkarnati/Desktop/os/week1skill
myshell> date
Sun Sep 27 11:18:01 IST 2026
myshell> hello123
exec: No such file or directory
Command could not be executed.
myshell> exit
Shell exiting...
vishalkarnati@VISHALS-MacBook-Air week1skill %
```

Task2:

To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios

```
UW PICO 5.09 File: error_shell.c Modified
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

#define MAX_INPUT 200
#define MAX_ARGS 20

/* Write an error message to error.log */
void log_error(const char *message)
{
    FILE *file = fopen("error.log", "a");

    if (file == NULL)
    {
        perror("Could not open error.log");
        return;
    }

    time_t current_time = time(NULL);

    fprintf(file,
            "[%s] %s\n",
            ctime(&current_time),
            message);

    fclose(file);
}

/* Parse command */
int parse_command(char *input, char *args[])
{
    int count = 0;

    char *token = strtok(input, " ");

    while (token != NULL && count < MAX_ARGS - 1)
    {
        args[count] = token;

        count++;

        token = strtok(NULL, " ");
    }

    args[count] = NULL;

    return count;
}

/* Execute command and detect errors */
void execute_command(char *args[])
{
    pid_t pid = fork();

    if (pid == -1)
    {
        log_error("fork failed");
        return;
    }

    if (pid == 0)
    {
        // Child process
        // ... (code for child process) ...
    }
    else
    {
        // Parent process
        // ... (code for parent process) ...
    }
}
```

GH Get Help
 GH Exit

GH WriteOut
 GH Justify

GH Read File
 GH Where is

GH Prev Pg
 GH Next Pg

GH Cut Text
 GH UnCut Text

GH Cur Pos
 GH To Spell

```
UW PICO 5.09 File: error_shell.c Modified
/* Parse command */
int parse_command(char *input, char *args[])
{
    int count = 0;

    char *token = strtok(input, " ");

    while (token != NULL && count < MAX_ARGS - 1)
    {
        args[count] = token;

        count++;

        token = strtok(NULL, " ");
    }

    args[count] = NULL;

    return count;
}

/* Execute command and detect errors */
void execute_command(char *args[])
{
    pid_t pid = fork();

    if (pid == -1)
    {
        perror("fork failed");
        log_error("fork() failed");
        return;
    }

    if (pid == 0)
    {
        execvp(args[0], args);

        /*
         * If execvp returns, execution failed.
         */
        perror("Command execution failed");
        exit(127);
    }

    int status;

    if (waitpid(pid, &status, 0) == -1)
    {
        perror("waitpid failed");
        log_error("waitpid() failed");
        return;
    }
}

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell
```

```
UW PICO 5.09 File: error_shell.c Modified

        log_error("fork() failed");
        return;
    }

    if (pid == 0)
    {
        execvp(args[0], args);

        /*
         * If execvp returns, execution failed.
         */
        perror("Command execution failed");
        exit(127);
    }

    int status;

    if (waitpid(pid, &status, 0) == -1)
    {
        perror("waitpid failed");
        log_error("waitpid() failed");
        return;
    }

    if (WIFEXITED(status))
    {
        int exit_status = WEXITSTATUS(status);

        if (exit_status != 0)
        {
            printf("Command failed with exit status %d\n",
                exit_status);
            log_error("Command returned a non-zero exit status");
        }
    }
    else if (WIFSIGNALED(status))
    {
        int signal_number = WTERMSIG(status);

        printf("Command terminated by signal %d\n",
            signal_number);
        log_error("Command terminated by a signal");
    }
}

/* Display the error log */
void show_log()
{
    FILE *file = fopen("error.log", "r");

    if (file == NULL)
    {
        return;
    }

    Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
    Exit Justify Where is Next Pg UnCut Text To Spell
```

```
UW PICO 5.09 File: error_shell.c Modified
{
    int exit_status = WEXITSTATUS(status);
    if (exit_status != 0)
    {
        printf("Command failed with exit status %d\n",
            exit_status);
        log_error("Command returned a non-zero exit status");
    }
}
else if (WIFSIGNALED(status))
{
    int signal_number = WTERMSIG(status);
    printf("Command terminated by signal %d\n",
        signal_number);
    log_error("Command terminated by a signal");
}
}

/* Display the error log */
void show_log()
{
    FILE *file = fopen("error.log", "r");

    if (file == NULL)
    {
        printf("No error log available.\n");
        return;
    }

    char line[300];
    printf("\n---- ERROR LOG ----\n");
    while (fgets(line, sizeof(line), file) != NULL)
    {
        printf("%s", line);
    }

    printf("-----\n");
    fclose(file);
}

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("Error Handling Shell\n");
    printf("Commands: logs, exit\n\n");

    while (1)
    {
        printf("myshell> ");

```

```
UW PICO 5.09 File: error_shell.c Modified
    return;
}

char line[300];
printf("\n---- ERROR LOG ----\n");
while (fgets(line, sizeof(line), file) != NULL)
{
    printf("%s", line);
}

printf("-----\n");
fclose(file);
}

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("Error Handling Shell\n");
    printf("Commands: logs, exit\n\n");

    while (1)
    {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0)
        {
            printf("Error: empty command\n");
            log_error("Empty command entered");
            continue;
        }

        if (strcmp(input, "exit") == 0)
        {
            printf("Shell exiting...\n");
            break;
        }

        if (strcmp(input, "logs") == 0)
        {
            show_log();
            continue;
        }
    }
}
```

```
UW PICO 5.09 File: error_shell.c Modified

if (fgets(input, sizeof(input), stdin) == NULL)
{
    break;
}

input[strcspn(input, "\n")] = '\0';

if (strlen(input) == 0)
{
    printf("Error: empty command\n");
    log_error("Empty command entered");
    continue;
}

if (strcmp(input, "exit") == 0)
{
    printf("Shell exiting...\n");
    break;
}

if (strcmp(input, "logs") == 0)
{
    show_log();
    continue;
}

int argc = parse_command(input, args);

if (argc == 0)
{
    printf("Error: invalid syntax\n");
    log_error("Invalid command syntax");
    continue;
}

execute_command(args);

return 0;
}

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell
```

```
vishalkarnati@VISHALS-MacBook-Air week11skill % nano error_shell.c
vishalkarnati@VISHALS-MacBook-Air week11skill % gcc error_shell.c -o error_shell
vishalkarnati@VISHALS-MacBook-Air week11skill % ./error_shell
Error Handling Shell
Commands: logs, exit

myshell> ls
error_shell      error_shell.c      integrated_shell      integrated_shell.c
myshell> abcxyz
Command execution failed: No such file or directory
Command failed with exit status 127
myshell> cat doesnotexist.txt
cat: doesnotexist.txt: No such file or directory
Command failed with exit status 1
myshell> logs
Command execution failed: No such file or directory
Command failed with exit status 127
myshell> logs

----- ERROR LOG -----
[Sun Sep 27 11:28:57 2026
] Command returned a non-zero exit status
[Sun Sep 27 11:21:28 2026
] Command returned a non-zero exit status
[Sun Sep 27 11:21:34 2026
] Command returned a non-zero exit status
-----
myshell> xit
Command execution failed: No such file or directory
Command failed with exit status 127
myshell> exit
Shell exiting...
vishalkarnati@VISHALS-MacBook-Air week11skill %
```

Task3:

To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings

```
UW PICO 5.89 File: memory_leak.c Modified
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    char *name;
    int *numbers;

    printf("Starting memory allocation test...\n");

    name = malloc(100 * sizeof(char));

    if (name == NULL)
    {
        printf("Memory allocation failed\n");
        return 1;
    }

    strcpy(name, "Operating Systems");

    printf("Name: %s\n", name);

    numbers = malloc(10 * sizeof(int));

    if (numbers == NULL)
    {
        printf("Memory allocation failed\n");
        free(name);
        return 1;
    }

    for (int i = 0; i < 10; i++)
    {
        numbers[i] = i * 10;
    }

    printf("Numbers:\n");

    for (int i = 0; i < 10; i++)
    {
        printf("%d ", numbers[i]);
    }

    printf("\n");

    /*
     * Intentionally NOT freeing:
     * * name
     * * numbers
     * * This creates memory leaks.
     */

    printf("Program finished\n");

    return 0;
}
```

Get Help Exit WriteOut Justify Read File Where is Prev Pg Next Pg Cut Text UnCut Text Cur Pos To Spell

```
vishalkarnati@VISHALS-MacBook-Air week11skill % nano memory_leak.c
vishalkarnati@VISHALS-MacBook-Air week11skill % gcc -g memory_leak.c -o memory_leak
vishalkarnati@VISHALS-MacBook-Air week11skill % ./memory_leak
Starting memory allocation test...
Name: Operating Systems
Numbers:
0 10 20 30 40 50 60 70 80 90
Program finished
vishalkarnati@VISHALS-MacBook-Air week11skill %
```

Task4:

To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports

```
UW PICO 5.09 File: test_redirect.sh Modified
#!/bin/bash

echo "=====
echo " I/O REDIRECTION TEST SUITE
echo "=====

passed=0
failed=0

echo ""
echo "Test 1: Input file exists"

if [ -f input.txt ]; then
    echo "PASS: input.txt exists"
    passed=$((passed + 1))
else
    echo "FAIL: input.txt does not exist"
    failed=$((failed + 1))
fi

echo ""
echo "Test 2: Run redirect program"

./redirect > test_output.txt

if [ $? -eq 0 ]; then
    echo "PASS: Program executed successfully"
    passed=$((passed + 1))
else
    echo "FAIL: Program execution failed"
    failed=$((failed + 1))
fi

echo ""
echo "Test 3: Check output file"

if [ -f output.txt ]; then
    echo "PASS: output.txt created"
    passed=$((passed + 1))
else
    echo "FAIL: output.txt not created"
    failed=$((failed + 1))
fi

echo ""
echo "Test 4: Compare input and output"

if cmp -s input.txt output.txt; then
    echo "PASS: Input and output contents match"
    passed=$((passed + 1))
else
    echo "FAIL: Input and output contents do not match"
    failed=$((failed + 1))
fi

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell
```

```
UW PICO 5.09 File: test_redirect.sh Modified

    echo "FAIL: Program execution failed"
    failed=$((failed + 1))
fi

echo ""
echo "Test 3: Check output file"

if [ -f output.txt ]; then
    echo "PASS: output.txt created"
    passed=$((passed + 1))
else
    echo "FAIL: output.txt not created"
    failed=$((failed + 1))
fi

echo ""
echo "Test 4: Compare input and output"

if cmp -s input.txt output.txt; then
    echo "PASS: Input and output contents match"
    passed=$((passed + 1))
else
    echo "FAIL: Input and output contents do not match"
    failed=$((failed + 1))
fi

echo ""
echo "Test 5: Output is not empty"

if [ -s output.txt ]; then
    echo "PASS: output.txt contains data"
    passed=$((passed + 1))
else
    echo "FAIL: output.txt is empty"
    failed=$((failed + 1))
fi

echo ""
echo "=====
echo " TEST REPORT
echo "=====

echo "Passed: $passed"
echo "Failed: $failed"

total=$((passed + failed))

echo "Total : $total"

if [ $failed -eq 0 ]; then
    echo ""
    echo "ALL TESTS PASSED"
    exit 0
else
    echo ""

```


